

HOMEWORK 2

LOGISTIC REGRESSION AND REGULARIZATION¹

CMU 10-701: MACHINE LEARNING (FALL 2024)

<https://piazza.com/cmu/fall2024/10701/home>

OUT: Thursday, Sept 19th, 2024

DUE: Wednesday, Oct 2nd, 2024, 11:59pm

START HERE: Instructions

- **Collaboration policy:** Collaboration on solving the homework is allowed, after you have thought about the problems on your own. It is also OK to get clarification (but not solutions) from books or online resources, again after you have thought about the problems on your own. There are two requirements: first, cite your collaborators fully and completely (e.g., “Jane explained to me what is asked in Question 2.1”). Second, write your solution *independently*: close the book and all of your notes, and send collaborators out of the room, so that the solution comes from you only. See the Academic Integrity Section in our [Course Syllabus](#) for more information.
- **Late Submission Policy:** See the late homework policy in our [Course Syllabus](#).
- **Submitting your work to Gradescope:** There will be two submission slots for this homework on Gradescope, the Written and the Programming:
 - For the written problems such as short answer, multiple choice, derivations, proofs, or plots, we will be using the written submission slot. Please use the provided template. The best way to format your homework is by using the Latex template released in the handout and writing your solutions in Latex. However submissions can be handwritten onto the template, but should be labeled and clearly legible. If your writing is not legible, you will not be awarded marks. Each derivation/proof should be completed in the boxes provided below the question, **you should not move or change the sizes of these boxes** as Gradescope is expecting your solved homework PDF to match the template on Gradescope. If you find you need more space than the box provides you should consider cutting your solution down to its relevant parts, if you see no way to do this, please add an additional page at the end of the homework and guide us there with a ‘See page xx for the rest of the solution’.
 - You are also required to upload your code, which you wrote to solve the final questions of this homework, to the Programming submission slot. Your code may

¹Compiled on Thursday 3rd October, 2024 at 22:51

be run by TAs so please make sure it is in a workable state.

- Regrade requests can be made after the homework grades are released, however this gives the TA the opportunity to regrade your entire paper, meaning if additional mistakes are found then points will be deducted.

For multiple choice or select all that apply questions, shade in the box or circle in the template document corresponding to the correct answer(s) for each of the questions. For $\text{\LaTeX}$ users, use $\blacksquare$ and $\bullet$ for shaded boxes and circles, and don't change anything else. If an answer box is included for showing work, **you must show your work!**

1 Regularized Logistic Regression [25 points]

In this question, we'll solve a small logistic regression problem by hand. (Later, we'll solve a larger problem with PyTorch code.) For this question, our data consists of two points in $\mathbb{R}^1$: $x_1 = 1, y_1 = 0, x_2 = 3, y_2 = 1$.

1. [7 points] Write out the loss function for this problem, using parameters w, b so that the linear part of the model is $wx + b$. Use the logistic (binary cross-entropy) loss. Use a Gaussian prior for w with mean 0 and standard deviation 1. Use no prior for b . (Equivalently, use a ridge (L_2) regularizer on w and no regularizer on b .) Omit additive constant terms (those that do not depend on w, b).

For binary cross-entropy (BCE) loss: $\mathcal{L} = y_t \log(1 + \exp(-z_t)) + (1 - y_t) \log(1 + \exp(z_t))$
 For $(x_1 = 1, y_1 = 0)$: $\mathcal{L}_1 = (0 \cdot \log(1 + \exp(-(w+b))) + (1-0) \cdot \log(1 + \exp(w+b))) = \log(1 + \exp(w+b))$
 For $(x_2 = 3, y_2 = 1)$: $\mathcal{L}_2 = (1 \cdot \log(1 + \exp(-(3w+b))) + (1-1) \cdot \log(1 + \exp(3w+b))) = \log(1 + \exp(-(3w+b)))$
Total loss with ridge regularization on w : $\mathcal{L} = \log(1 + \exp(w+b)) + \log(1 + \exp(-3w-b)) + \frac{1}{2}w^2$

2. [6 points] Write out the derivative of the loss function with respect to b . Use the shorthand $\sigma(z) = \frac{1}{1+\exp(-z)}$. At the optimal value of w, b , this derivative will equal zero; use this fact to derive a linear relationship between the optimal w and b .

$$\frac{d}{db}[\mathcal{L} = \log(1 + \exp(w+b)) + \log(1 + \exp(-3w-b)) + \frac{1}{2}w^2]$$

$$\frac{d\mathcal{L}}{db} = \frac{1}{1 + \exp(w+b)} \cdot \exp(w+b) + \frac{1}{1 + \exp(-3w-b)} \cdot \exp(-3w-b) \cdot (-1)$$

$$\frac{d\mathcal{L}}{db} = \frac{e^{w+b}}{1 + e^{w+b}} - \frac{e^{-3w-b}}{1 + e^{-3w-b}} = \sigma(w+b) - \sigma(-3w-b)$$

$$\frac{d\mathcal{L}}{db} = 0 \implies \sigma(w+b) = \sigma(-3w-b) \implies w+b = -3w-b \implies 4w = -2b \implies b = -2w$$

Solution — Derivative: $\frac{d\mathcal{L}}{db} = \sigma(w+b) - \sigma(-3w-b)$ **Linear relationship:** $b = -2w$

3. [6 points] Write out the derivative of the loss function with respect to w . At the optimum, this derivative will also equal zero. Use this fact, along with the answer to the previous part, to derive an equation in just w .

$$\frac{d}{dw}[\mathcal{L} = \log(1 + \exp(w+b)) + \log(1 + \exp(-3w-b)) + \frac{1}{2}w^2]$$

$$\frac{d\mathcal{L}}{dw} = \frac{1}{1 + \exp(w+b)} \cdot \exp(w+b) + \frac{1}{1 + \exp(-3w-b)} \cdot \exp(-3w-b) \cdot (-3) + w$$

we have that $b = -2w$ from above:

$$\frac{d\mathcal{L}}{dw} = \frac{e^{-w}}{1 + e^{-w}} - \frac{3e^{-w}}{1 + e^{-w}} + w = \sigma(-w) - 3\sigma(-w) + w = w - 2\sigma(-w) = 0 \implies w = \frac{2}{e^w + 1}$$

Solution — Derivative: $\frac{d\mathcal{L}}{dw} = w - 2\sigma(-w)$ **Equation:** $w = 2\sigma(-w) = \frac{2}{e^w + 1}$

4. **[6 points]** Use any combination of pencil, paper, calculator, and computer to solve for the optimal w and b to at least three significant figures. For example, one strategy is binary search on w using the equation derived in the previous part.

Approach: find the intersection of the two previous equations using a graphing calculator. We have that

$$(1) \quad w = -\frac{b}{2} \quad \text{and} \quad (2) \quad w = \frac{2}{e^w + 1}$$

The intersection is expressed by the following:

$$-\frac{b}{2} = \frac{2}{e^w + 1}$$

We find that the optimal pair $(w, b) = (0.6748, -1.350)$

2 Convexity of Logistic Regression [25 points]

As above, consider logistic regression for predicting a label y given an input $x \in \mathbb{R}^d$. In logistic regression, we model the log-odds as a linear function of the data and find weights to maximize the likelihood of our data under the resulting model. For this question we'll take y to be in $\{-1, +1\}$ for simplicity, instead of $\{0, 1\}$.

2.1 Convex Optimization

Recall that the negative conditional log-likelihood for a logistic regression model can be written as

$$\mathcal{L}(w) = -\log P(y \mid x^{1:T}, w) = \sum_{i=1}^T \log(1 + \exp(-y_i w^\top x_i)).$$

Our goal is to find the weight vector w that minimizes $\mathcal{L}$. Unfortunately, for this objective, we cannot derive a closed-form expression for the minimizer; instead, we use gradient descent or SGD. We know these gradient methods will converge to the optimal value of w when $\mathcal{L}$ is convex. Recall that a real-valued function $f : S \rightarrow \mathcal{R}$, defined on a convex set S , is said to be *convex* if

$$f(tx_1 + (1-t)x_2) \leq tf(x_1) + (1-t)f(x_2) \quad \forall x_1, x_2 \in S, \forall t \in [0, 1].$$

In this question, you will prove that $\mathcal{L}$ is indeed a convex function (and hence, the conditional log likelihood is a concave function).

1. [4 points] Given n convex functions, $f_1, f_2, \dots, f_n$, show that a linear combination $\sum_{i=1}^n a_i f_i(x)$ is also convex for any nonnegative weights $a_i \geq 0$.

Let $g(x) = \sum_{i=1}^n a_i f_i(x)$. For $x_1, x_2 \in S$ and $t \in [0, 1]$, consider the following:

$$g(tx_1 + (1-t)x_2) = \sum_{i=1}^n a_i f_i(tx_1 + (1-t)x_2)$$

We are given that each f_i ($f_1, f_2, \dots, f_n$) is convex, so:

$$f_i(tx_1 + (1-t)x_2) \leq tf_i(x_1) + (1-t)f_i(x_2) \implies a_i f_i(tx_1 + (1-t)x_2) \leq a_i (tf_i(x_1) + (1-t)f_i(x_2))$$

$$\implies \sum_{i=1}^n a_i f_i(tx_1 + (1-t)x_2) \leq \sum_{i=1}^n a_i (tf_i(x_1) + (1-t)f_i(x_2))$$

Applying linearity of summation: $g(tx_1 + (1-t)x_2) \leq t \sum_{i=1}^n a_i (f_i(x_1)) + (1-t) \sum_{i=1}^n a_i (f_i(x_2))$
Applying the definition of g , we have that $g(tx_1 + (1-t)x_2) \leq tg(x_1) + (1-t)g(x_2)$, i.e., g is convex.
Thus, we have shown that the linear combination of n convex functions, $g(x) = \sum_{i=1}^n a_i f_i(x)$, is also convex.

2. [4 points] Another property of twice differentiable convex functions is that the second derivative is non-negative. Using this property, show that $f(x) = \log(1 + \exp x)$ is a convex function. Note that this property is both sufficient and necessary, i.e., if $f''(x)$ exists, then $(f''(x) \geq 0 \text{ everywhere}) \iff f \text{ is convex}$.

Let us compute the first derivative:

$$\frac{d}{dx}[\log(1 + \exp x)] = \frac{1}{1 + \exp x} \cdot \exp x = \frac{\exp x}{1 + \exp x}$$

So, $f'(x) = \frac{\exp x}{1 + \exp x}$. Now, we will compute the second derivative (using the quotient rule):

$$f''(x) = \frac{(1 + \exp x)(\exp x) - (\exp x)(\exp x)}{(1 + \exp x)^2} = \frac{\exp x + (\exp x)^2 - (\exp x)^2}{(1 + \exp x)^2} = \frac{\exp x}{(1 + \exp x)^2}$$

So, $f''(x) = \frac{\exp x}{(1 + \exp x)^2}$

We know that $\exp x > 0 \quad \forall x \in \mathbb{R}$ and that $(1 + \exp x)^2 > 0 \quad \forall x \in \mathbb{R}$. So,

$$f''(x) = \frac{\exp x}{(1 + \exp x)^2} \geq 0 \quad \forall x \in \mathbb{R}$$

Since $f''(x) \geq 0$ everywhere, then $f(x) = \log(1 + \exp x)$ is a convex function.

3. [4 points] Let $f_i : \mathcal{S} \rightarrow \mathcal{R}$ for $i = 1, \dots, n$ be a list of convex functions. Is $f(x) = \max_i f_i(x)$ also convex? If yes, prove it. If not, provide a counterexample.

We are given that $f(x) = \max_i f_i(x)$. So, by definition of f :

$$f(tx_1 + (1 - t)x_2) = \max_i f_i(tx_1 + (1 - t)x_2)$$

Since each f_i is convex, we know that:

$$f_i(tx_1 + (1 - t)x_2) \leq tf_i(x_1) + (1 - t)f_i(x_2)$$

Thus, $\max_i f_i(tx_1 + (1 - t)x_2) \leq \max_i (tf_i(x_1) + (1 - t)f_i(x_2))$

Taking the maximum of the sum of two convex functions is less than or equal to the sum of their individual maximums. So,

$$\max_i f_i(tx_1 + (1 - t)x_2) \leq t \max_i f_i(x_1) + (1 - t) \max_i f_i(x_2)$$

Applying the definition of f gives us:

$$f(tx_1 + (1 - t)x_2) \leq tf(x_1) + (1 - t)f(x_2)$$

The inequality holds for all $x_1, x_2 \in S$ and $t \in [0, 1]$, so $f(x) = \max_i f_i(x)$ is convex.

4. [3 points] Suppose f is convex and g is linear. Show that $f(g(x))$ is convex.

Since g is linear, then $g(x) = ax + b$ for some $a, b \in \mathbb{R}$.

Let $h(x) = f(g(x)) = f(ax + b)$, $a, b \in \mathbb{R}$

Consider $g(tx_1 + (1-t)x_2) = a(tx_1 + (1-t)x_2) + b = tg(x_1) + (1-t)g(x_2)$

Now, consider the following:

$$h(tx_1 + (1-t)x_2) = f(g(tx_1 + (1-t)x_2)) = f(tg(x_1) + (1-t)g(x_2))$$

Since f is convex, we know that for $g(x_1) = y_1$ and $g(x_2) = y_2$, the following holds:

$$f(ty_1 + (1-t)y_2) \leq tf(y_1) + (1-t)f(y_2)$$

Since $y_1 = g(x_1)$ and $y_2 = g(x_2)$: $f(tg(x_1) + (1-t)g(x_2)) \leq tf(g(x_1)) + (1-t)f(g(x_2))$. So:

$$h(tx_1 + (1-t)x_2) = f(g(tx_1 + (1-t)x_2)) \leq tf(g(x_1)) + (1-t)f(g(x_2)) = th(x_1) + (1-t)h(x_2)$$

Thus, we have that $h(x) = f(g(x))$ is convex.

5. [10 points] Show that the negative log likelihood $\mathcal{L}$ is a convex function. You may use the conclusions from previous parts.

The negative log likelihood, $\mathcal{L}(w) = \sum_{i=1}^T \log(1 + \exp(-y_i w^\top x_i))$, is a sum of terms in the following form:

$$f_i(w) = \log(1 + \exp(-y_i w^\top x_i))$$

In part 2, we showed that the function $f(x) = \log(1 + \exp x)$ is convex. In this case, we see that the original argument, x , has been replaced with $-y_i w^\top x_i$

The function $g(x) = -y_i w^\top x_i$ is linear in w , since it consists of an inner product (multiplication and addition), scaled by a constant ($-y_i$).

From part 4, we know that if $f(x)$ is convex and if $g(x)$ is linear, then $f(g(x)) = f(-y_i w^\top x_i) = \log(1 + \exp(-y_i w^\top x_i))$ is convex.

So, since $f(x) = \log(1 + \exp x)$ is convex and $g(x) = -y_i w^\top x_i$ is linear, we have:

$$f_i(w) = \log(1 + \exp(-y_i w^\top x_i))$$

and $f_i(w)$ is convex for each i .

The negative log likelihood is the sum of convex functions:

$$\mathcal{L}(w) = \sum_{i=1}^T f_i(w)$$

Since the sum of convex functions is convex by part 1, it follows that $\mathcal{L}(w)$ is convex.

Thus, $\mathcal{L}(w)$ is a convex function.

3 Logistic Regression for Sentiment Classification [50 Points]

For the programming portion of this assignment, you will explore fitting a binary logistic regression classifier to predict the sentiment of Yelp reviews using text embeddings and PyTorch. Sections 3.1 and 3.2 describe the Python notebook you'll be working with. Your deliverables for the programming portion are listed in Section 3.3 below; be sure to copy all of the deliverables from the notebook to your PDF hand-in so that you receive credit for them. Also be sure to upload your code to Gradescope.

3.1 Vector Arithmetic with Text Embeddings

In this problem, you will be using the Yelp Polarity dataset², a modified subset of the Yelp Reviews dataset³. The Yelp Reviews dataset contains English-language user reviews of businesses and their respective star rating on a scale of 1 to 5. The Yelp Polarity dataset takes $\sim 600k$ samples from the Yelp Reviews dataset and adjusts their labels such that reviews with 1 or 2 stars are categorized as negative (label “0”) and reviews with 4 or 5 stars are categorized as positive (label “1”). Reviews with 3 stars are excluded. The reviews themselves are unchanged between the Yelp Reviews and Yelp Polarity datasets.

The language model you will use to generate embeddings of these reviews is a small BERT (Bidirectional Encoder Representations from Transformers⁴) model which we load from Hugging Face, an online hub for open-source models. This model has a corresponding tokenizer which we use to transform strings of text into numerical sequences. The BERT model then takes the output sequences of the tokenizer and embeds them into 128-dimensional vectors; these vectors will later become the input features for our logistic regression. By using a large pretrained language model, we are able to take advantage of a significantly larger vocabulary and more expressive features than if we were to train a text embedding model ourselves.

Your first task is to complete some helper functions which you'll use throughout the programming assignment. Then, you'll examine the embeddings of some randomly sampled reviews, as well as the embeddings of the same reviews which have been manually “flipped” such that their sentiment has shifted: positive words/phrases have been changed to comparable negative ones and vice versa. (For reproducibility, we have provided these randomly sampled reviews and their flipped versions.)

Then, you'll use vector arithmetic to find the difference between the embeddings of the original and flipped reviews; these difference vectors capture how much flipping the sentiment of a review impacted the magnitude and direction of its embedding. Lastly, you will use cosine similarity to calculate the alignment between difference vectors to see if flipping the

²https://huggingface.co/datasets/fancyzhx/yelp_polarity

³https://huggingface.co/datasets/Yelp/yelp_review_full

⁴<https://arxiv.org/abs/1810.04805>

sentiment of multiple reviews in the same way (positive to negative or negative to positive) causes shifts in the same direction in the embedding space.

The goal of this section is to examine how semantic changes to the original data manifest in the embedding/feature space. By changing only the sentiment of each review and leaving the rest of their context relatively untouched, we attempt to focus solely on one component of the semantic information captured by the embedding model.

3.2 Training a Logistic Regression Classifier

In this section, you will be using the same Yelp Polarity dataset and the same embedding model to train a binary logistic regression classifier.

Using PyTorch, you will walk through the model training pipeline: defining a custom Dataset class, creating a model class, writing a training loop, incorporating regularization, and evaluating your model's performance. The logistic regression model will use the review embeddings as features, so it will learn a 128-dimensional weight vector. The starter code provides the training, validation, and test sets as well as training parameters.

After training, you will produce a plot of training and validation losses per epoch using stored values and then evaluate your model's performance on a held-out test set.

3.3 Deliverables

Please complete the questions below. In order to get credit you **MUST** upload both this PDF and your code to the appropriate Gradescope submission slots under Homework 2.

1. **[10 points]** We only examined the cosine similarity values between difference vectors rather than directly between embeddings. Given that we know BERT is context-aware (in other words, the influence of a word on the embedding depends on surrounding words in the input text), why might it be less informative to compare whole embeddings rather than their differences? (Hint: is sentiment the only information that is encoded in an embedding? How might two embeddings with opposite sentiments still be very similar?)

It may be less informative to compare whole embeddings rather than their differences because BERT embeddings encode much more than just sentiment; they also capture syntax, semantics, and context. Because of this, two sentences with opposite sentiments can still have very similar embeddings if they share topics, structure, or vocabulary. So, comparing whole embeddings can obscure differences in sentiment because other shared information may dominate.

However, comparing difference vectors may be more informative because the differences isolate how sentiment changes between two texts, making it easier to focus on the sentiment shift without being distracted by other similarities in the embeddings.

2. **[10 points]** If we needed to train a higher-accuracy sentiment classifier but were restricted to logistic regression classifiers like this one, what part of the model development pipeline (e.g., the dataset, feature engineering, training loop, etc.) could we try changing to improve test accuracy?

Each of the examples mentioned (the dataset, feature engineering, training loop, etc.) are all parts of the model development pipeline that could be tuned to improve the test accuracy.

In terms of the dataset, we could increase the size of the dataset, either by collecting more samples or through data augmentation. More training examples may help to increase test accuracy. Additionally, we could ensure similar numbers of positive and negative examples in the training set to ensure balanced classes, and so that the model does not overfit to one majority class.

Next, we could also make some improvements in feature engineering. We could try implementing the commonly used Term Frequency-Inverse Document Frequency (TF-IDF) technique for language processing to see if this helps to improve test accuracy.

Finally, in terms of training the model, we could also experiment with different hyperparameters, e.g., batch size, learning rate, etc. to try and improve the test accuracy.

Overall, there are several factors we could change to try and improve the test accuracy, including making changes to the dataset (augmenting or balancing), feature engineering (TF-IDF approach), or tuning the training hyperparameters.

3. [10 points] Using the data in the provided `miniYelpReviewsDataset_dictionary.pkl` file, train your model from scratch using the following hyperparameters:

- Learning rate = 0.001
- SGD momentum = 1e-5
- Batch size = 32
- L2 regularization $\lambda = 0.01$
- Epochs = 200

Use the starter code in the cell labeled “Q3.3.3 Deliverable” to plot training and validation loss curves and include an image of your plot below. Additionally, report the accuracy and loss your model achieves on the test set.

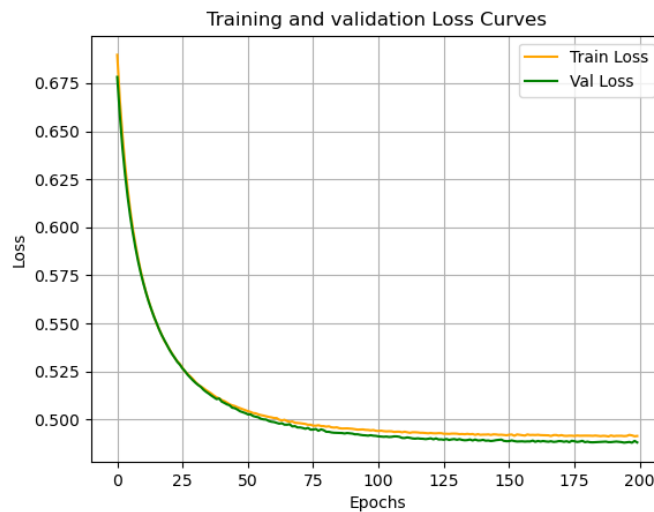


Figure 1: Training and validation loss curves

Test set accuracy: 0.8040

Test set loss (avg loss): 0.4902

4. [7 points] Using the model you trained in Q3.3.3, run the cell labeled “Q3.3.4 Deliverable” and save an image of both heatmaps generated by the code. Please include those images below. Additionally, report the first 5 weights of your trained classifier. (These are for grading purposes to ensure correct implementation.)

Refer to the cosine similarity values computed in the notebook cell labeled “Compare model weights to difference vectors”. What do you observe about the relationship between the difference vectors from the flipped reviews section and the model’s learned weights? Does the learned weight vector point more in the direction of the positive difference vectors or the negative difference vectors? Why would you expect the difference vectors to be aligned with / not aligned with the classifier weights?

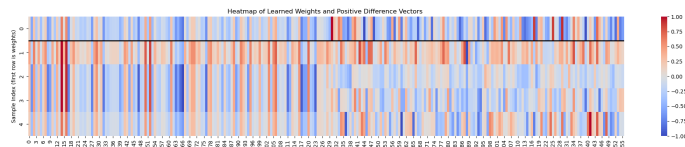


Figure 14: Heatmap of Learned Weights and Positive Difference Vectors

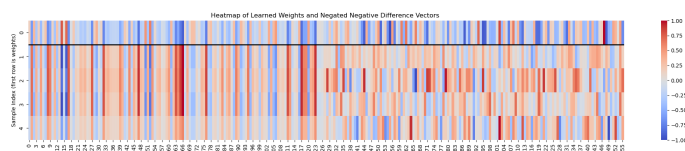


Figure 15: Heatmap of Learned Weights and Negated Negative Difference Vectors

First five weights: 0.1426404 -0.23373175 0.23214473 -0.08923489 0.14186089

For the cosine similarity values, we see a positive cosine similarity between the model weights and the positive difference vectors and a negative cosine similarity between the model weights and negative difference vectors. These values indicate that the learned weight vector points more in the direction of the positive difference vectors (indicated by a positive cosine similarity). This implies that the model is more aligned with features that characterize positive reviews, whereas the negative cosine similarity for the negative difference vectors indicates that the model tends to distinguish negative reviews by opposing these feature directions. Overall, since the model was trained to separate positive and negative reviews, it makes sense that the learned weight vector aligns with the features driving positive sentiment while opposing those driving negative sentiment.

5. [7 points] Did you observe overfitting in the training process? Explain why this was the case by considering factors such as model complexity, dataset size, and regularization techniques. If overfitting was not observed, discuss whether it would be possible to adjust the conditions (for example, extended training time or changes in model architecture) to eventually cause overfitting.

Overfitting was not observed, as evidenced by similar accuracy values for training, validation, and test sets, and overfitting is unlikely in this model due to the use of L2 regularization, model simplicity, and training time.

It would be possible to adjust the model to cause overfitting. Some examples of how this could be achieved would be to reduce the regularization strength, as the model would be penalized less for large weights. Reducing regularization could lead to a more complex model prone to overfitting.

Additionally, the training time could be extended so that the model has an opportunity to memorize the training data and overfit, which would manifest in a plateau of validation accuracy.

Finally, we could reduce the size of the dataset. With fewer examples, the model would be more prone to overfitting.

So, although it appears that overfitting did not occur, there are changes that could be made, e.g., relaxing regularization, increasing training time, and reducing dataset size, that could increase the potential for overfitting to occur.

6. **[6 points]** Refer to the section of the notebook titled “Compare model weights to difference vectors”. In this section, we used the trained model to perform inference on our flipped review embeddings. We observed that on the flipped embeddings, the model makes more mistakes than on original, un-flipped review embeddings. Why do you think this is the case?

The model makes more mistakes on the flipped embeddings than the original, un-flipped embeddings because the model wasn't trained on flipped reviews. The model was trained on the original, un-flipped reviews, meaning that the weights were updated based on the patterns and distributions from the original, un-flipped embeddings. When the review is flipped, the embeddings become less aligned with the learned sentiment patterns, leading to poorer predictions by the model.

4 Collaboration Questions

1. (a) Did you receive any help whatsoever from anyone in solving this assignment?

Solution Yes

- (b) If you answered ‘yes’, give full details (e.g. “Jane Doe explained to me what is asked in Question 3.4”)

Solution Ellie Escobar-Robledo clarified what was being asked in question 2.3.

2. (a) Did you give any help whatsoever to anyone in solving this assignment? **Solution**

Yes

- (b) If you answered ‘yes’, give full details (e.g. “I pointed Joe Smith to section 2.3 since he didn’t know how to proceed with Question 2”)

Solution I explained the concept being described in question 1 to Ellie Escobar-Robledo.

3. (a) Did you find or come across code that implements any part of this assignment?

Solution No.

- (b) If you answered ‘yes’, give full details (book & page, URL & location within the page, etc.).

Solution N/A